| | |
|---|---|
| **Batch: B2** | **Roll No.: 16010124107** |
| **Experiment No. 9** | |
| **Grade: AA / AB / BB / BC / CC / CD /DD** | |
| **Signature of the Staff In-charge with date** | |

Title Implementation of Disk scheduling algorithms - FCFS,SSTF,SCAN, CSCAN

---

**AIM:** Implementation of Disk scheduling algorithms - FCFS, SSTF, SCAN, CSCAN

---

**Expected Outcome of Experiment:**

**CO 4.** To understand various Memory, I/O and File management techniques.

---

**Books/ Journals/ Websites referred:**

1. **Silberschatz A., Galvin P., Gagne G. "Operating Systems Principles", Willey Eight edition.**
2. **Achyut S. Godbole , Atul Kahate "Operating Systems", McGraw Hill Third Edition.**
3. **Sumitabha Das " UNIX Concepts & Applications", McGraw Hill Second Edition.**

---

**Algorithm completed by:**

A) **Roll No: 16010124107 : FCFS and CSCAN**
B) **Roll No: 16010124100: SSTF and SCAN**

**Pre Lab/ Prior Concepts:**

The main purpose of the disk scheduling algorithm is to select a disk request from the queue of IO requests and decide the schedule when this request will be processed.
Goal of Disk Scheduling Algorithm

**FCFS Scheduling Algorithm**

It is the simplest Disk Scheduling algorithm. It services the IO requests in the order in which they arrive. There is no starvation in this algorithm, every request is serviced.

**SSTF Scheduling Algorithm**

Shortest seek time first (SSTF) algorithm selects the disk I/O request which requires the least disk arm movement from its current position regardless of the direction. It reduces the total seek time as compared to FCFS.
It allows the head to move to the closest track in the service queue.

**Scan Algorithm**

It is also called the Elevator Algorithm. In this algorithm, the disk arm moves into a particular direction till the end, satisfying all the requests coming in its path, and then it turns back and moves in the reverse direction satisfying requests coming in its path.
It works in the way an elevator works, elevator moves in a direction completely till the last floor of that direction and then turns back.

**Look Scheduling**

It is like the SCAN scheduling algorithm to some extent except for the difference that, in this scheduling algorithm, the arm of the disk stops moving inwards (or outwards) when no more request in that direction exists. This algorithm tries to overcome the overhead of SCAN algorithm which forces disk arm to move in one direction till the end regardless of knowing if any request exists in the direction or not.

---

**Assigned Algorithm 1 details:** Ashwera

**FCFS**

Step 1: Take input of disk requests.

Step 2: Take initial head position.

Step 3: Initialize total seek time = 0.

Step 4: Start from first request.

Step 5: Calculate distance = |current head − request|.

Step 6: Add distance to total seek time.

Step 7: Move head to that request.

Step 8: Repeat for all requests.

Step 9: Output total seek time.

**CSCAN:**

Step 1: Take disk requests and head position.

Step 2: Choose direction (left or right).

Step 3: Sort all requests.

Step 4: Move head in chosen direction.

Step 5: Service all requests on the way.

Step 6: Go till disk end.

Step 7: Jump to the other end (no servicing).

Step 8: Continue in the same direction.

Step 9: Service remaining requests.

Step 10: Calculate total seek time.

**<u>Assigned Algorithm 2 details:</u>** Riya

**SSTF :**

Step 1 : Input array of disk request and initial head position.

Step 2 : Find the positive distance from the head position of the all disk requests.

Step 3 : Find the track which has not been accessed yet & has the minimum distance from head.

Step 4 : Increment total seek count & make the most recent track the new head.

Step 5 : Continue until all requests have been accessed.

Step 6 : Calculate total seek time.

**SCAN :**

Step 1 : Input array of disk request and initial head position.

Step 2 : Let direction represent whether the head is moving towards left or right.

Step 3 : Increment total seek count & make the most recent track the new head.

Step 4 : Service all tracks at once in the direction the head is moving.

Step 5 : Continue until we reach one of the ends of the disk.

Step 6 : Reverse the direction and service all tracks in that direction.

Step 7 : Continue until all requests have been accessed.

Step 8 : Calculate the total seek time.

**<u>Source code:</u>**

FCFS:

```
#include <bits/stdc++.h>

using namespace std;


signed main() {

    cout << "Enter number of processes and head \n";

    int n, head;

    cin >> n >> head;

    vector<int> req(n);

    cout << "Enter disk requests\n";

    for(int i = 0; i < n; i++) cin >> req[i];

    int totalSeek = 0;


    for(int i = 0; i < n; i++) {

        totalSeek += abs(req[i] - head);

        head = req[i];

    }

    cout << "Total Seek Time: " << totalSeek << endl;

}
```

CSCAN:

```
#include <bits/stdc++.h>

using namespace std;


signed main() {
```

```
    cout << "Enter number of processes and head\n";
    int n, head;
    cin >> n >> head;
    cout << "Enter direction\n";
    char dir; cin >> dir;
    vector<int> req(n);
    cout << "Enter disk requests\n";
    for(int i = 0; i < n; i++) cin >> req[i];
    int disk_size;
    cin >> disk_size;

    req.push_back(0);
    req.push_back(disk_size - 1);
    sort(req.begin(), req.end());

    int totalSeek = 0;
    auto it = lower_bound(req.begin(), req.end(), head);

    if(dir == 'R') {
        for(auto i = it; i != req.end(); i++) {
            totalSeek += abs(*i - head);
            head = *i;
        }
        totalSeek += abs((disk_size - 1) - head);
        head=0;
```

```
        totalSeek += (disk_size - 1);

        for(auto i = req.begin(); i != it; i++) {
            totalSeek += abs(*i - head);
            head = *i;
        }

    } else {
        for(auto i = it - 1; i >= req.begin(); i--) {
            totalSeek += abs(*i - head);
            head = *i;
        }
        totalSeek += abs(head - 0);
        head = disk_size - 1;

        totalSeek += (disk_size - 1);

        for(auto i = it; i != req.end(); i++) {
            totalSeek += abs(*i - head);
            head = *i;
        }
    }
```

```
    cout << "Total Seek Time: " << totalSeek << endl;

}
```

SSTF :

```
#include <iostream>
#include <vector>
#include <cmath>
#include <climits>
using namespace std;

int main() {
    int n, head;
    cout << "Enter number of requests: ";
    cin >> n;

    vector<int> req(n);
    cout << "Enter requests: ";
    for(int i = 0; i < n; i++) cin >> req[i];

    cout << "Enter initial head position: ";
    cin >> head;

    vector<bool> visited(n, false);
    int totalSeek = 0;

    cout << "\nSeek Sequence: ";

    for(int i = 0; i < n; i++) {
        int idx = -1;
        int minDist = INT_MAX;

        // find closest request
        for(int j = 0; j < n; j++) {
            if(!visited[j]) {
                int dist = abs(req[j] - head);
```

```
                if(dist < minDist) {
                    minDist = dist;
                    idx = j;
                }
            }
        }

        // move head
        totalSeek += minDist;
        head = req[idx];
        visited[idx] = true;

        cout << head << " ";
    }

    cout << "\nTotal Seek Time: " << totalSeek << endl;

    return 0;
}
```

SCAN :

```
#include<bits/stdc++.h>
using namespace std;

int main() {
    int n, head, disk_size, direction;

    cout << "Enter number of requests: ";
    cin >> n;

    vector<int> requests(n);

    cout << "Enter the requests:\n";
    for (int i = 0; i < n; i++) {
        cin >> requests[i];
    }

    cout << "Enter initial head position: ";
```

```
    cin >> head;

    cout << "Enter disk size: ";
    cin >> disk_size;

    cout << "Enter direction (0 for left, 1 for right): ";
    cin >> direction;

    vector<int> left, right;

    // Separate requests into left and right of head
    for (int i = 0; i < n; i++) {
        if (requests[i] < head)
            left.push_back(requests[i]);
        else
            right.push_back(requests[i]);
    }

    // Add boundary depending on direction
    if (direction == 0)
        left.push_back(0);
    else
        right.push_back(disk_size - 1);

    // Sort both sides
    sort(left.begin(), left.end());
    sort(right.begin(), right.end());

    int seek_count = 0;
    vector<int> seek_sequence;


    if (direction == 1) {
        // Move right first
        for (int i = 0; i < right.size(); i++) {
            seek_sequence.push_back(right[i]);
            seek_count += abs(head - right[i]);
            head = right[i];
        }
```

```
        // Then move left
        for (int i = left.size() - 1; i >= 0; i--) {
            seek_sequence.push_back(left[i]);
            seek_count += abs(head - left[i]);
            head = left[i];
        }
    } else {
        // Move left first
        for (int i = left.size() - 1; i >= 0; i--) {
            seek_sequence.push_back(left[i]);
            seek_count += abs(head - left[i]);
            head = left[i];
        }

        // Then move right
        for (int i = 0; i < right.size(); i++) {
            seek_sequence.push_back(right[i]);
            seek_count += abs(head - right[i]);
            head = right[i];
        }
    }

    cout << "\nSeek Sequence: ";
    for (int i = 0; i < seek_sequence.size(); i++) {
        cout << seek_sequence[i] << " ";
    }

    cout << "\nTotal Seek Time: " << seek_count << endl;

    return 0;
}
```

**Output screenshots:**

FCFS:

```
Enter number of processes and head
8 53
Enter disk requests
98 183 37 122 14 124 65 67
Total Seek Time: 640
PS C:\Users\syeda\OneDrive\Desktop\semiv>
```

SSTF:

```
PS C:\Users\KJSCE\Desktop\os exp>  & 'c:\Use
n=Microsoft-MIEngine-In-chxqzvru.lix' '--std
-Pid-rbpji4yy.u2p' '--dbgExe=C:\msys64\mingw
Enter number of requests: 8
Enter requests: 176 79 34 60 92 11 41 114
Enter initial head position: 50

Seek Sequence: 41 34 11 60 79 92 114 176
Total Seek Time: 204
PS C:\Users\KJSCE\Desktop\os exp>
```

SCAN:

```
Enter number of requests: 8
Enter the requests:
176 79 34 60 92 11 41 114
Enter initial head position: 50
Enter disk size: 199
Enter direction (0 for left, 1 for right): 0

Seek Sequence: 41 34 11 0 60 79 92 114 176
Total Seek Time: 226
```

CSCAN:

```
PS C:\Users\syeda\OneDrive\Desktop\semiv> cd
File } ; if ($?) { .\tempCodeRunnerFile }
Enter number of processes and head
8 53
Enter direction
R
Enter disk requests
98 183 37 122 14 124 65 67
200
Total Seek Time: 382
PS C:\Users\syeda\OneDrive\Desktop\semiv>
```

**Conclusion:**

FCFS is the simplest disk scheduling algorithm and serves requests in the order they arrive, but it results in high seek time and poor performance. SSTF improves performance by selecting the nearest request, reducing seek time, but it may cause starvation of distant requests. SCAN provides better fairness by moving the head in one direction and then reversing, ensuring all requests are served. C-SCAN further improves uniformity by servicing requests in one direction only, providing more consistent waiting time.

**Post Lab Descriptive Questions**

i. Explain how SSTF can lead to starvation. Can this problem occur in SCAN or LOOK? Justify your answer.

SSTF (Shortest Seek Time First) selects the disk request closest to the current head position, which minimizes seek time but can lead to starvation. Starvation occurs when requests that are far from the current head position keep getting postponed because new requests keep arriving closer to the head. As a result, distant requests may wait indefinitely and may never get serviced.

In contrast, SCAN and LOOK do not suffer from starvation. In SCAN, the disk arm moves in one direction servicing all requests along the path and then reverses direction, ensuring that every request is eventually serviced. Similarly, LOOK moves in a direction only up to the last request in that direction before reversing. Since both algorithms systematically traverse the disk in an ordered manner, no request is ignored indefinitely, and thus starvation does not occur.

ii. How does the size of the disk (number of tracks) influence the performance of the SCAN, C-SCAN, and LOOK algorithms? Provide examples to support your answer.
Disk size affects head movement in scheduling algorithms. In SCAN and C-SCAN, larger disks increase seek time because the head travels to the disk ends, causing more delay. In LOOK, the head moves only up to the last request, so performance is less affected by disk size, making it more efficient.

iii. Suppose that a disk drive has 200 cylinders numbered 0 to 199. Consider a disk queue with I/O requests on the following cylinders in their arriving order: 54, 95, 73, 130, 25, 48, 110, 30, 45, 180, and 190. The disk head is assumed to be at cylinder 24. Starting from the current head position, what is the total distance (in cylinders) the disk arm moves to satisfy all the pending requests for the SCAN and LOOK disk-scheduling algorithm?

**Date: 15/04/2026**